

Polytech OS User – TP partie 3

I – Présentation

Ce TP est la suite de la partie 2 qui contient déjà tous les descriptifs du protocole **BEUIP** (BEUI over IP) et de la librairie **creme** (Commandes Rapides pour l'Envoi de Messages Evolués) qui ont permis à notre programme **biceps** (Bel Interpréteur de Commandes des Elèves de Polytech Sorbonne) de posséder des commandes internes pour gérer les échanges de messages. Il est donc indispensable d'avoir le texte de la partie 2 à portée de vue.

II – Objectif

Le but de ce TP est de réaliser un protocole en s'inspirant du fonctionnement très simple de NetBEUI, et de l'utiliser dans une application qui va permettre à chacun de communiquer avec les autres étudiants, ceci en utilisant les bibliothèques aux standards POSIX qui ont été définies pour les systèmes de la famille Unix, dont GNU/Linux fait partie.

Et nous en profiterons pour vous faire aussi découvrir quelques outils développés dans le projet GNU, créé en 1983 par Richard Stallman (cf : <https://www.gnu.org/home.fr.html>).

Il est conseillé de relire les exemples de codes expliqués en cours et les supports de cours sur la Programmation Système et Réseau sous Unix. Vous avez également à votre disposition le code d'un interpréteur de commandes simple avec moins de 90 lignes de C (cf ! Triceps.tar.gz).

Tous ces éléments sont sur le site de l'association Getall : <https://cours.getall.fr/polytech/> .

III - Le sujet

Amélioration du protocole **BEUIP** (BEUI over IP) et de la librairie **creme** (Commandes Rapides pour l'Envoi de Messages Evolués) qui permet déjà à notre programme **biceps** (Bel Interpréteur de Commandes des Elèves de Polytech Sorbonne) de posséder des commandes internes pour gérer les échanges de messages.

L'objectif des premières étapes est de vous guider pour créer ces améliorations.

1 – Etape 1 : Utilisation du multi-threading

Nous avons ajouté à notre **biceps**, dans la seconde partie du TP, un serveur UDP pour échanger des messages avec les autres acteurs du réseau. Ce serveur a été créé en tant que processus fils de **biceps**. Et donc **biceps** ne peut accéder à la liste des utilisateurs présents, celle-ci étant dans l'espace d'adressage virtuel du processus serveur, qui lui est inaccessible.

Il nous a fallu utiliser une astuce : nous envoyer des messages à nous-même pour communiquer. Ce qui ouvre la porte à des failles de sécurité comme une attaque du type « *man-in-the-middle* ».

Un pirate qui enverrait un message avec, dans l'entête IP, l'adresse de l'expéditeur égale à '127.0.0.1', pourrait donner des ordres à notre place comme envoyer de faux messages, ou même arrêter notre serveur !

On pourrait, pour parer à cela, communiquer avec des *pipes* internes. Mais nous allons faire mieux en utilisant le fait que le multi-threading nous permet de faire cohabiter l'interpréteur de commandes et le serveur UDP dans le même processus, et donc de pouvoir se partager les mêmes variables globales !

AVERTISSEMENTS :

- nous utiliserons exclusivement la librairie POSIX 1003.1 présentée en cours,
- l'écriture du code va demander beaucoup plus d'attention surtout dans les parties qui accéderont aux données partagées par plusieurs threads. Je vous renvoie au cours sur les sémaphores et les mutex.

1.1 Mise en place du thread serveur UDP

Il suffit de fabriquer une fonction du type :

```
void * serveur_udp(void * p)
```

à laquelle on passera en paramètre le pseudo de l'utilisateur et qui exécutera le code du serveur UDP. Cette fonction sera utilisée dans le corps de la fonction correspondant à la commande interne « **beuip start pseudo** » pour lancer le thread qui sera le serveur UDP.

La structure des messages ainsi que le port restent les mêmes que dans le second TP (cf : TP 2)

RAPPEL IMPORTANT : Le premier octet (*octet1*) du message d'identification est une clé au format ASCII. Ses valeurs possibles correspondent donc aux caractères '0', '1', etc.

1.2 Modification des commandes non sécurisées

Nous avons bien compris que **biceps** a maintenant accès à la table des couples (nom + adresse). Et donc les datagrammes pour les commandes suivantes :

- liste (octet1 = '3'),
- message à un pseudo (octet1 = '4'),
- message à tout le monde (octet1 = '5'),

n'ont plus lieu d'être puisqu'on peut les réaliser directement avec des instructions internes au processus, ce qui renforce la sécurité en empêchant les attaques évoquées au début de ce texte.

Pour cela nous allons fabriquer une seule fonction qui va regrouper tout le code nécessaire :

```
void commande(char octet1, char * message, char * pseudo)
```

ATTENTION :

- le serveur UDP ne gère plus que les valeurs '0', '1', '2' et '9' de l'octet1, et va signaler les erreurs pour détecter les éventuelles tentatives de piratage,
- il faut bien penser à gérer les accès concurrents à la table entre le serveur UDP qui la met à jour et la fonction **commande** qui l'utilise en lecture,
- pour la commande interne « **beuip stop** » (arrêt du serveur UDP) il faut réfléchir à une méthode simple qui va permettre d'envoyer un message avec octet1 = '0' à toute la liste et d'arrêter le thread sans terminer le processus,
- bien vérifier qu'on ne peut faire « **beuip start** » qu'une seule fois et que l'on ne peut faire « **beuip stop** » que si le serveur UDP est actif,

- afin que tout le monde soit en phase, les **datagrammes** avec octet1 égal à '0' ou '9' seront sans AR.

2 – Etape 2 : Adaptation automatique aux réseaux présents

Nous utilisons pendant le TP le réseau offert par le routeur wifi dont l'ESSID est TPOSUSER qui nous offre une adresse IP appartenant au réseau de classe C : 192.168.88.0.

L'idée est de pouvoir continuer d'utiliser notre **biceps** sur d'autres réseaux. Le serveur UDP ne va plus utiliser l'adresse fixe '192.168.88.255' mais les adresses *broadcast* des interfaces présentes et valides au moment du lancement du serveur.

2.1 Modification des messages « broadcast »

Nous allons donc modifier le code du serveur UDP qui va automatiquement détecter les interfaces opérationnelles, récupérer les adresses de **broadcast** pour envoyer, si cette adresse est différente de l'adresse *localhost* (127.0.0.1), le message avec l'octet1 à '1' et le pseudo.

Pour cela nous nous pencherons sur les fonctions **getifaddrs()** (cf: **man 3 getifaddrs**) et **getnameinfo()** (cf: **man 3 getnameinfo**). Le premier man contient un exemple complet d'utilisation qui doit permettre de réaliser rapidement le code nécessaire.

CONSEILS :

- Il faut se concentrer sur les éléments dont (*sa_family == AF_INET*),
- Un **getnameinfo()** sur *ifa->ifa_broadaddr* suffit pour recueillir l'adresse **broadcast** !

2.2 Modification de la table des contacts

Suite à cela nous pouvons avoir à gérer plusieurs réseaux et la table de dimension fixe, qui était une manière de commencer rapidement les tests, n'est plus adaptée. Nous allons donc la remplacer par une liste chaînée en utilisant la structure suivante :

```
/* le chainage des couples (pseudo, IPV4) */
#define LPSEUDO 23          /* longueur maxi du pseudo */
struct elt {                /* structure d'un element */
    char nom[LPSEUDO+1];    /* nom de l'element */
    char adip[16];          /* IPv4 de l'element */
    struct elt * next;      /* adresse du prochain element */
};
```

Il faut faire ensuite trois fonctions. Deux utilisées par le serveur UDP :

```
void ajouteElt(char * pseudo, char * adip);
void supprimeElt(char * adip);
```

et une utilisée par le thread initial *main()* pour la commande interne « **mess liste** » :

```
void listeElts(void);
```

La fonction **ajouteElt()** va maintenir la liste par ordre alphabétique croissant sur le pseudo pour faciliter la recherche et vérifier plus rapidement si un pseudo existe.

3 – Etape 3 : Création de commandes pour échanger des fichiers

L'idée est maintenant de proposer aux autres utilisateurs un ensemble de fichiers dont ils vont pouvoir consulter la liste et télécharger depuis leur **biceps**.

Pour cela il faut commencer par créer un sous répertoire public, par exemple **pub**, dans lequel on va mettre des fichiers quelconques (images, pdf, textes, etc) pour les proposer. Le nom du répertoire est au choix de chacun. Nous l'appellerons *reppub* dans la suite du texte.

3.1 Création d'un thread serveur TCP

Comme nous l'avons vu en cours, une connexion TCP nous offre une interface qui permet d'échanger des flux de données dans les deux sens.

Nous allons donc créer une seconde fonction :

```
void * serveur_tcp(void * rep)
```

à laquelle on passera en paramètre le nom du répertoire qui contient les fichiers à partager et qui exécutera le code du serveur TCP qui fera un **bind()** toujours sur le port **9998**. Cette fonction sera utilisée également dans le corps de la fonction correspondant à la commande interne « **beuip start pseudo** » pour lancer le thread qui sera le serveur TCP. Nous aurons donc deux threads serveurs.

Il est conseillé de relire les sources du programme **servtcp** présenté en cours le 4 mars.

ATTENTION : Maintenant la commande interne « **beuip stop** » doit également stopper le thread serveur TCP.

3.2 Commande « beuip ls pseudo »

Cette commande a pour but de demander au serveur TCP de l'utilisateur **pseudo** de nous envoyer la liste des fichiers contenu dans son répertoire **reppub**.

Pour cela nous allons mettre en place deux fonctions.

Une première : `void demandeListe(char * pseudo)`

qui va faire un **connect()** (**man 2 connect**) sur le serveur situé au port 9998 de l'adresse IP du **pseudo**, et lui envoyer, avec un **write()**, un octet '**L**' pour lui signifier sa demande.

Elle va ensuite attendre la réponse dans une boucle de **read()** et afficher sur la sortie standard (**stdout**) le contenu du buffer jusqu'à l'obtention de **EOF (End of File)**.

Elle fermera ensuite la connexion avant de se terminer.

Cette fonction sera utilisée par le thread **main()** pour exécuter la commande interne.

Une seconde : `void envoiContenu(int fd)`

qui récupère en paramètre le *file descriptor* obtenu par l'**accept()** du serveur TCP (**man 2 accept**).

Cette fonction commence à faire un **read()** d'un octet.

Si cet octet est '**L**' elle crée un processus fils qui va exécuter la commande '**ls -l reppub/**' en prenant bien soin, avant de faire l'**exec()**, de rediriger **stdout** et **stderr** sur **fd** !

Cette fonction sera utilisée par le serveur TCP, soit directement dans le cas pas très performant d'un serveur « factotum », soit en créant un thread pour être plus réactif.

3.3 Commande « beuip get pseudo nomfic »

Cette commande a pour but de demander au serveur de l'utilisateur **pseudo** de nous envoyer le fichier **nomfic** contenu dans son répertoire public.

Tout est déjà quasiment en place pour faire cela !

Il suffit de décider que si, dans la fonction **envoiContenu()**, le premier octet est '**F**', alors il faut lire ensuite le nom du fichier qui se termine par '**\n**'. Puis créer un processus fils qui va exécuter la

commande 'cat reppub/nomfichier' en prenant bien soin, avant de faire l'**exec()**, de rediriger **stdout** et **stderr** sur **fd** !

Il n'y a plus qu'à faire la fonction : void demandeFichier(char * pseudo, char * nomfic) qui va faire un **connect()** (**man 2 connect**) sur le serveur situé au port 9998 de l'adresse IP du **pseudo**, et lui envoyer, avec un **write()**, une chaîne de caractères commençant par '**F**' suivit du nom du fichier et terminée par un '\n' pour lui signifier sa demande.

Elle va ensuite attendre la réponse dans une boucle de **read()** et sauvegarder dans reppub/nomfic le contenu du buffer jusqu'à l'obtention de **EOF (End of File)**.

Elle fermera ensuite la connexion avant de se terminer.

Cette fonction sera utilisée par le thread **main()** pour exécuter la commande interne.

ATTENTION : il faut penser à effectuer au minimum quelques contrôles. Par exemples :

- quand le fichier distant demandé n'existe pas,
- quand le fichier demandé distant a le même nom qu'un fichier local déjà présent,
- et peut-être d'autres.

3.4 Propositions d'améliorations

Ce système est loin d'être parfait, vous vous en êtes sans doute rendu compte. Voici quelques propositions pour l'améliorer, si le coeur vous en dit :

- Quand un message arrive, le thread serveur UDP fait l'équivalent d'un **printf()** pour en informer l'utilisateur. Et quelques fois cela peut tomber comme un cheveu sur la soupe car celui-ci est en train de taper une commande. Il faudrait donc stocker le message via un pipe, une liste chaînée, ou autres. Il y a plusieurs moyens possibles. Et afficher les messages en attente juste avant de présenter le prompt pour la saisie d'une nouvelle commande,
- Faire en sorte de pouvoir utiliser l'adresse IP à la place du pseudo si celui-ci est utilisé par plusieurs utilisateurs,
- Ajouter dans le répertoire **reppub** un sous répertoire du nom du pseudo distant, ou de son adresse IP, afin de différencier des fichiers qui ont le même nom mais qui proviennent d'utilisateurs différents,
- Ajouter une fonction de recherche « **beuip find nomfic** » qui parcourt la liste des utilisateurs en essayant de télécharger le fichier nomfic,
- Toutes les données transitent en clair sur le réseau, il faudrait donc ajouter du chiffrement,
- Etc ... votre imagination va vous dicter la suite. Amusez-vous !

Comme dans les TP's précédents, afin de bien profiter de votre programme **biceps v3**, ajouter des instructions de compilation conditionnelles pour n'inclure les messages qui servent à tracer l'exécution du code que si on le demande à la compilation avec l'option **-DTRACE**.

On peut même avoir plusieurs niveaux de traces : **TRACE1**, **TRACE2**, etc.

Bravo, vous venez de réaliser la **version 3** de notre programme **biceps** !

Fin du troisième TP.